

Detailed Notes on I/O Techniques and Bus Communication

1 Bus

Definition: A **bus** is a shared communication pathway that connects components like the CPU, memory, and I/O devices for data transfer.

Types of Buses:

- **Data Bus:** Carries actual data.
- **Address Bus:** Carries memory or I/O addresses.
- **Control Bus:** Carries control signals (e.g., Read/Write).

Analogy: Think of a bus as a *highway* where cars (data, address, control signals) travel between cities (CPU, memory, I/O).

2 Bus Arbitration

Definition: Bus arbitration is the process of deciding which device gets control of the bus when multiple devices request access.

Methods:

- **Daisy Chaining**
- **Polling**
- **Independent Request**

Analogy: Imagine multiple students wanting to talk to a teacher. Arbitration decides whose turn it is.

3 Daisy Chaining

Definition: Devices are connected in a series. The CPU sends an interrupt acknowledgment that passes through each device.

Priority: The device closest to the CPU has the highest priority.

Advantages: Simple and cheap.

Disadvantages: Fixed priority and delay for low-priority devices.

Analogy: Think of people standing in a line; the teacher talks to the first person who raised their hand and ignores others until that's done.

4 Polling

Definition: CPU repeatedly checks each device to see if it needs service.

Advantages: Simple to implement.

Disadvantages: Wastes CPU time if devices are not ready.

Analogy: Like a waiter checking every table repeatedly asking “Need anything?” even if customers don’t call.

5 Independent Request

Definition: Each device has its own dedicated request line to the CPU.

Advantages: Faster response and easier priority implementation.

Disadvantages: Needs more hardware and is less scalable.

Analogy: Each student in a class has a separate buzzer to signal the teacher.

6 Programmed I/O (PIO)

Definition: CPU is responsible for all data movement between memory and I/O using explicit instructions.

Types:

- **Synchronous (Blocking):** CPU waits for device to be ready.
- **Asynchronous (Non-blocking):** CPU does other work while I/O is pending.

Analogy: A person manually pouring water from one container to another without stopping.

7 Memory-Mapped I/O

Definition: I/O devices are assigned memory addresses, and CPU accesses them like regular memory.

Advantages: Easy to use with standard instructions.

Disadvantages: Occupies memory address space.

Analogy: Treating a vending machine as if it’s a shelf in your kitchen—you can reach in and grab or place items just like memory.

8 I/O-Mapped I/O (Isolated I/O)

Definition: I/O devices have separate address space. CPU uses special instructions like IN and OUT.

Advantages: Full memory space remains available.

Disadvantages: Requires special I/O instructions.

Analogy: Having a separate mailbox just for utility bills (I/O), different from your regular mail (memory).

9 Direct Memory Access (DMA)

Definition: A special controller transfers data between memory and I/O without CPU involvement.

Steps:

1. CPU configures DMA controller.
2. DMA transfers data.
3. DMA notifies CPU on completion.

Modes: Burst, Cycle Stealing, Transparent.

Advantages: High-speed and efficient.

Disadvantages: Hardware complexity and potential bus contention.

Analogy: Like hiring a mover to transfer boxes between rooms while you focus on cooking.

10 Interrupts

Definition: A signal to the CPU to pause current execution and service an event.

Types:

- Hardware / Software
- Maskable / Non-maskable
- Vectored / Non-vectored

Interrupt Handling Steps:

1. CPU finishes current instruction.
2. Saves context.
3. Jumps to ISR (Interrupt Service Routine).
4. Executes ISR.
5. Restores context and resumes.

Advantages: Efficient and responsive.

Disadvantages: Complexity and potential for interrupt overload.

Analogy: Like your phone ringing while you study—pause, answer the call (handle interrupt), then resume studying.